

EXPERIMENT - 4

Title

Joins, Subqueries, EXISTS, Simulated INTERSECT and EXCEPT, and EXPLAIN

Aim

To perform different SQL operations on the Employee-Department-Project schema using INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, simulated INTERSECT and EXCEPT, and to compare query execution plans using EXPLAIN.

Objectives

1. To understand and implement INNER JOIN.
2. To implement LEFT JOIN.
3. To perform a self-join on the Employee table.
4. To perform a 3-way join between Employee, Department, and Project.
5. To understand and implement correlated subqueries.
6. To use EXISTS for checking the existence of records.
7. To simulate INTERSECT using SQL conditions.
8. To simulate EXCEPT using NOT EXISTS.
9. To analyze query execution plans using EXPLAIN.

Software/ Tools Required

- ByteXL / Nimbus
- MySQL
- SQL

Theory

A **JOIN** is used to combine data from two or more tables based on a related column. Joins are very useful when related information is stored in different tables.

In this experiment, the Employee, Department, and Project tables are used to retrieve related information.

Aryan Pratap Singh
25LBCS3266

1. INNER JOIN

An **INNER JOIN** returns only those records that have matching values in both tables.

For example, Employee and Department can be joined using Dept ID.

2. LEFT JOIN

A **LEFT JOIN** returns all records from the left table and the matching records from the right table. If there is no matching record, NULL values are returned for the right table.

3. Self-Join

A **self-join** is a join in which a table is joined with itself. It is useful for comparing records within the same table.

For example, employees having a higher salary than another employee can be compared using a self-join.

4. Three-Way Join

A **three-way join** combines three tables in a single query. In this experiment, Employee, Department, and Project are joined to display complete employee information.

5. Correlated Subquery

A correlated subquery is a subquery that depends on the value of the outer query. The subquery is evaluated for each row processed by the outer query.

6. EXISTS

The EXISTS operator checks whether a subquery returns at least one record. It returns TRUE when the subquery produces a result.

7. Simulated INTERSECT

INTERSECT returns records common to two result sets. When direct INTERSECT is not used, the same result can be simulated using EXISTS or a suitable JOIN.

8. Simulated EXCEPT

EXCEPT returns records present in the first result set but not present in the second result set. It can be simulated using NOT EXISTS .

9. EXPLAIN

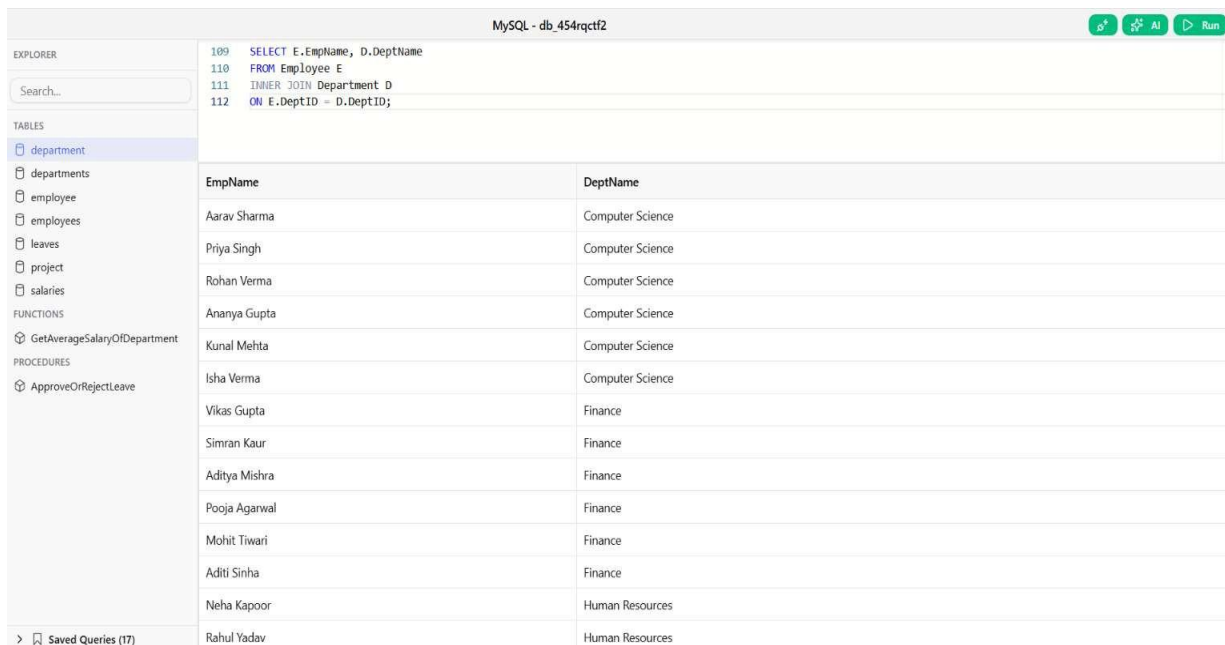
EXPLAIN is used to display the execution plan of an SQL query. It provides information about how MySQL plans to execute the query, including tables used, indexes, access type, and estimated rows.

SQL Implementation

1. INNER JOIN

Question: Display employee names along with their department names.

```
SELECT E.EmpName, D.DeptName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID;
```



The screenshot shows a MySQL IDE interface with a query editor and a results table. The query editor contains the following SQL code:

```
109 SELECT E.EmpName, D.DeptName
110 FROM Employee E
111 INNER JOIN Department D
112 ON E.DeptID = D.DeptID;
```

The results table displays the following data:

EmpName	DeptName
Aarav Sharma	Computer Science
Priya Singh	Computer Science
Rohan Verma	Computer Science
Ananya Gupta	Computer Science
Kunal Mehta	Computer Science
Isha Verma	Computer Science
Vikas Gupta	Finance
Simran Kaur	Finance
Aditya Mishra	Finance
Pooja Aganwal	Finance
Mohit Tiwari	Finance
Aditi Sinha	Finance
Neha Kapoor	Human Resources
Rahul Yadav	Human Resources

E- → C t: app.txt.aVnimbuj454rqrtf2

MySQL - db_454rqrtf2

109 SELECT E.EmpName, o.deptName

110 FROM Employee E

111 INNER JOIN department D

112 ON E.DeptID = D.DeptID;

Search...

(*) dt-partm_cm

El deparments

e employee

e employee

B leave<1

El project

B salaries

ii) GetAverageSalaryOfDepartment

(i) ApproveOrRejectLeave

Amit Kumar

Jshita Roy

Yash Thakur

Nisha Sharma

Arjun Singh

Riya Malhotra

Varun Saxena

Tanya Banerjee

Harsh Vardhan Dev Patel

Meera Joshi

Sahil Khan

Kcr-yasNair

M.mishra

Human Resources

Human Resources

Human Resources

Marketing

Marketing

Marketing

Marketing

Marketing

Research and Development

Research and Development

Research and Development

Research and Development

Research and Development

> Q Saved Queries (17)

2. LEFT JOIN

Question: Display all employees and their department names.

```
SELECT E.EmpName, D.DeptName
FROM Employee E
LEFT JOIN Department D
ON E.DeptID = D.DeptID;
```

MySQL - db_454nctft

114 SELECT E.EmpName, D.DeptName

115 FROM Employee E

116 LEFT JOIN Department D

117 ON E.DeptID = D.DeptID;

Search...

f3 dt-firmment

El deparments

El employee

!J employee,

El leaves

tJ pmject

e salaries

FUNCTIONS

© GetAverageSalaryOfDepartment1

PROCEDURB

tr) ApproveOrRejectLeave

EmpName

Aarav Sharma

Priya Singh

Rohan Verma

Ananya Gupta

Kuoal Mehta

Neha Kapoor

Rahul Yadav

Sneha Jain

Amit Kumar

Jshita Roy

Vikas Gupta

Simran Kaur

Aditya Mishra

Pooja Agarwal

DeptName

Computer Science

Computer Science

Computer Science

Computer Science

Computer Science

Human Resources

Human Resources

Human Resources

Human Resources

Finance

Finance

Finance

Finance

> Q Saved Queries (11)

Aryan Pratap Singh
25LBCS3266

app.bytext.ai/nimbus/454rqctf2

MySQL - db_454rqctf2

```

114 SELECT E.EmpName, D.DeptName
115 FROM Employee E
116 LEFT JOIN Department D
117 ON E.DeptID = D.DeptID;

```

EmpName	DeptName
Nisha Sharma	Marketing
Arjun Singh	Marketing
Riya Malhotra	Marketing
Varun Saxena	Marketing
Tanya Bansal	Marketing
Dev Patel	Research and Development
Meera Joshi	Research and Development
Sahil Khan	Research and Development
Kavya Nair	Research and Development
Manish Jain	Research and Development
Isha Verma	Computer Science
Yash Thakur	Human Resources
Aditi Sinha	Finance
Harsh Vardhan	Marketing
Diva Arora	Research and Development

3. Self-Join

Question: Find pairs of employees where one employee has a higher salary than another employee.

```

SELECT E1.EmpName AS Employee1,
       E1.Salary AS Salary1,
       E2.EmpName AS Employee2,
       E2.Salary AS Salary2
FROM Employee E1 JOIN Employee E2
ON E1.Salary > E2.Salary;

```

Note: This query can produce many rows because every possible salary comparison is considered.

MySQL - db_454rqctf2

```

118 SELECT E1.EmpName AS Employee1,
119        E1.Salary AS Salary1,
120        E2.EmpName AS Employee2,
121        E2.Salary AS Salary2
122 FROM Employee E1
123 JOIN Employee E2
124 ON E1.Salary > E2.Salary;

```

Employee1	Salary1	Employee2	Salary2
Dev Patel	75000.00	Aarav Sharma	72000.00
Mohit Tiwari	73000.00	Aarav Sharma	72000.00
Diya Agarwal	69000.00	Priya Singh	65000.00
Kavya Nair	70000.00	Priya Singh	65000.00
Meera Joshi	66000.00	Priya Singh	65000.00
Dev Patel	75000.00	Priya Singh	65000.00
Tanya Bansal	67000.00	Priya Singh	65000.00
Mohit Tiwari	73000.00	Priya Singh	65000.00
Vikas Gupta	68000.00	Priya Singh	65000.00
Aarav Sharma	72000.00	Priya Singh	65000.00
Diya Agarwal	69000.00	Rohan Verma	58000.00
Aditi Sinha	64000.00	Rohan Verma	58000.00
Kavya Nair	70000.00	Rohan Verma	58000.00

```

119  SELECT E1.LmpNam,e AS E1qYee1,
      e1.">-ala.y AS sala.y.1,
111      E2.EmpName AS E111p111yee2,
      In      E1.Salc1ry AS Salary7
      FROM C111p111yee E1
      JOIN E111p111yee E2
      ON E1.Salary > E2.Salary;
125

```

MoeraJoshi	66000.00	Sahil Ichhan	58000.00
Tanya Bansal	75000.00	Sahil Khann	58000.00
Ilyar M.rhico	60000.00	Sahil Khan	80000.00
MohitThwal	73000.00	Sahil Khcm	55000.00
Aditya Mish,a	59000.00	Sahil Khan	58000.00
	6200000	Sahil Khan	5800000
	68000.00	Sahil Khan	58000.00
	60000.00	Sahil Khan	58000.00
A=mv" Guptli	61100.00	Sahil Khan	58000.00
PTlyrlSingh	65000.00	Sahil Khlin	80000.00
Aarr,vSM,ma	72000.00		58000.00
DlloPatei	75000.00		
Mohit Thwad	73000.00		70000.00

[illegible]

C
E; opp byte)(LaVnImbus/4S4rqcfZ
*
0
•
I

	MySQL - db_4S4rqcfZ		
d-partm, 11	1111		
T1 departmenil	119 SELECT ELE111priante AS Employeeet,		
e employee	120 E1.salary AS Salary),		
f employees	111 {2.{111puivee AS C-r,loyee2,		
f lcv"	111 C2.salary AS salary2		
f project	123 FROM 6Employee H		
e Hlities	11+1 JOIN (-plyee E2		
@ Ge1AwrageSalaryOfOep,1,Iment	125 00 (1.Salary > CLSalary;		
PHOCIDURII			
@ ApproveO,Rkjc:ctleave			
	Simmn KCal,le	62000.00	Ynsh Thnkur
	VikasGupta	68000.00	Yash Thakur
	Ishita Roy	57000.00	Yash Thaku1
	Sneha Jain	6000000	Yash Thakur
	Rihul visav	52000.00	Y11shThnkur
	ICund Mehta	55000.00	Yash Thakur
	Ananya Gupta	61000.00	Yash Thakv1
	Rohan Verma	57000.00	Y11sh Thakur
	PriyitiSingh	6500000	Ynsh Thakur
	AartNShormn	72000.00	Y11shThlkur
	Diya Agarwal	69000.00	Adili Sinha
	Kavya Nair	70000.00	Adlli Sinha
	Mce,-a->oshl	66000.00	Aditi Sinha
	Dev Patel	75000.00	Adltf Sinha

> Saved Queries (17)

Aryan Pratap Singh
25LBCS3266

MySQL - db_454rqctf2

```

118
119 SELECT E1.EmpName AS Employee1,
120        E1.Salary AS Salary1,
121        E2.EmpName AS Employee2,
122        E2.Salary AS Salary2
123 FROM Employee E1
124 JOIN Employee E2
125 ON E1.Salary > E2.Salary;

```

Tanya Bansal	67000.00	Harsh Vardhan	57000.00
Riya Malhotra	63000.00	Harsh Vardhan	57000.00
Mohit Tiwari	73000.00	Harsh Vardhan	57000.00
Aditya Mishra	59000.00	Harsh Vardhan	57000.00
Simran Kaur	62000.00	Harsh Vardhan	57000.00
Vikas Gupta	68000.00	Harsh Vardhan	57000.00
Sneha Jain	60000.00	Harsh Vardhan	57000.00
Ananya Gupta	61000.00	Harsh Vardhan	57000.00
Rohan Verma	58000.00	Harsh Vardhan	57000.00
Priya Singh	65000.00	Harsh Vardhan	57000.00
Aarav Sharma	72000.00	Harsh Vardhan	57000.00
Kavya Nair	70000.00	Diya Agarwal	69000.00
Dev Patel	75000.00	Diya Agarwal	69000.00
Mohit Tiwari	73000.00	Diya Agarwal	69000.00
Aarav Sharma	72000.00	Diya Agarwal	69000.00

4. Three-Way Join

Question: Display employee name, department name, project name, and salary.

```

SELECT E.EmpName,
       D.DeptName,
       P.ProjectName,
       E.Salary
FROM Employee E
INNER JOIN Department DON  E.DeptID = D.DeptID
INNER JOIN Project PON    E.ProjectID = P.ProjectID;

```

5. Correlated Subquery

Question: Display employees whose salary is greater than the average salary of their own department.

```

SELECT E.EmpName,
       E.Salary,
       E.DeptID
FROM Employee E
WHERE E.Salary > (
    SELECT AVG(E2.Salary)
    FROM Employee E2
    WHERE E2.DeptID = E.DeptID
);

```

MySQL - db_454rqctf2

```

126 SELECT E.Employee,
127        E.Salary,
128        E.DeptID
129 FROM Employee E
130 WHERE E.Salary > (
131        SELECT AVG(E2.Salary)
132        FROM Employee E2
133        WHERE E2.DeptID = E.DeptID
134 )
135

```

Aarav Sharma	72000.00	101
Priya Singh	65000.00	101
Ananya Gupta	61000.00	101
Sneha Jain	60000.00	102
Ishita Roy	57000.00	102
Vikas Gupta	68000.00	103
Mohit Tiwari	73000.00	103
Riya Malhotra	63000.00	104
Tanya Bansal	67000.00	104
Dev Patel	75000.00	105
Meera Joshi	66000.00	105
Kavya Nair	70000.00	105
Aditi Sinha	64000.00	103
Diya Agarwal	69000.00	105

6. EXISTS

Question: Display departments that have at least one employee.

```

SELECT D.DeptID,
       D.DeptName
FROM Department D
WHERE EXISTS (
    SELECT 1
    FROM Employee E
    WHERE E.DeptID = D.DeptID
)

```

MySQL - db_454rqctf2

```

146 SELECT D.DeptID,
147        D.DeptName
148 FROM Department D
149 WHERE EXISTS (
150        SELECT 1
151        FROM Employee E
152        WHERE E.DeptID = D.DeptID
153 )

```

DeptID	DeptName
101	Computer Science
103	Finance
102	Human Resources
104	Marketing
105	Research and Development

7. Simulated INTERSECT

Question: Find departments that have employees and are also associated with employees earning more than 70000.

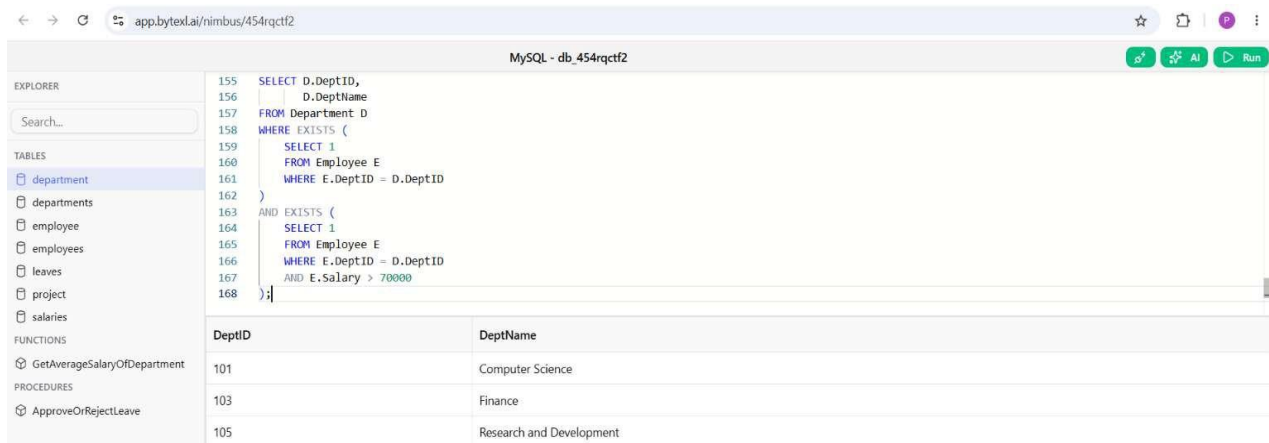
Aryan Pratap Singh
25LBSCS3266

```

SELECT D.DeptID,
       D.DeptName
FROM Department D
WHERE EXISTS (
    SELECT 1
    FROM Employee E
    WHERE E.DeptID = D.DeptID
)
AND EXISTS (
    SELECT 1
    FROM Employee E
    WHERE E.DeptID = D.DeptID
    AND E.Salary > 70000
);

```

This returns departments common to both conditions and therefore simulates the intersection of the two result sets.



The screenshot shows a MySQL IDE interface. The top bar indicates the database is 'MySQL - db_454rqctf2'. The left sidebar shows a tree view of database objects, including tables like 'department', 'departments', 'employee', 'employees', 'leaves', 'project', and 'salaries'. The main editor displays a SQL query that matches the one in the first block. Below the query editor, the results are shown in a table with two columns: 'DeptID' and 'DeptName'. The results table contains three rows of data.

DeptID	DeptName
101	Computer Science
103	Finance
105	Research and Development

8. Simulated EXCEPT

Question: Find departments that have employees but do not have any employee earning more than 70000.

```

SELECT D.DeptID,
       D.DeptName
FROM Department D
WHERE EXISTS (
    SELECT 1

```

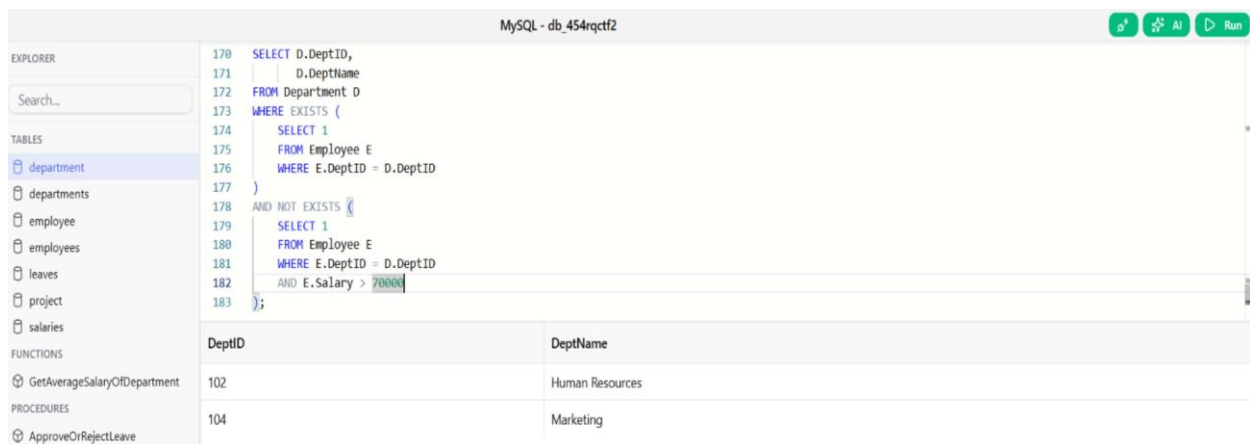
Aryan Pratap Singh
25LBCS3266

```

        FROM Employee E
        WHERE E.DeptID = D.DeptID
    )
    AND NOT EXISTS (
        SELECT 1
        FROM Employee E
        WHERE E.DeptID = D.DeptID
        AND E.Salary > 70000
    );

```

This returns departments satisfying the first condition but not the second condition and therefore simulates EXCEPT.



The screenshot shows a MySQL IDE window titled "MySQL - db_454rqctf2". The left sidebar contains an "EXPLORER" panel with a search bar and a list of tables: department, departments, employee, employees, leaves, project, salaries. The main editor displays a SQL query with line numbers 170 to 183. The query is as follows:

```

170 SELECT D.DeptID,
171        D.DeptName
172 FROM Department D
173 WHERE EXISTS (
174     SELECT 1
175     FROM Employee E
176     WHERE E.DeptID = D.DeptID
177 )
178 AND NOT EXISTS (
179     SELECT 1
180     FROM Employee E
181     WHERE E.DeptID = D.DeptID
182     AND E.Salary > 70000
183 );

```

Below the query editor, the results are displayed in a table with two columns: DeptID and DeptName. The results are as follows:

DeptID	DeptName
102	Human Resources
104	Marketing

9. EXPLAIN - INNER JOIN

To examine the execution plan of the INNER JOIN:

```

EXPLAIN
SELECT E.EmpName, D.DeptName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID;

```

MySQL - db_454rqctf2												
EXPLORER Search... TABLES department departments employee employees leaves project salaries FUNCTIONS GetAverageSalaryOfDepartment												
<pre> 185 EXPLAIN 186 SELECT E.EmpName, D.DeptName 187 FROM Employee E 188 INNER JOIN Department D 189 ON E.DeptID = D.DeptID; </pre>												
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
	1	SIMPLE	D	null	index	PRIMARY	DeptName	202	null	5	100	Using index
	1	SIMPLE	E	null	ref	DeptID	DeptID	5	db_454rqctf2.D.DeptID	6	100	null

10. EXPLAIN - LEFT JOIN

EXPLAIN

```

SELECT E.EmpName, D.DeptName
FROM Employee E
LEFT JOIN Department D
ON E.DeptID = D.DeptID;

```

MySQL - db_454rqctf2												
EXPLORER Search... TABLES department departments employee employees leaves project salaries FUNCTIONS GetAverageSalaryOfDepartment												
<pre> 191 EXPLAIN 192 SELECT E.EmpName, D.DeptName 193 FROM Employee E 194 LEFT JOIN Department D 195 ON E.DeptID = D.DeptID; </pre>												
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
	1	SIMPLE	E	null	ALL	null	null	null	null	30	100	null
	1	SIMPLE	D	null	eq_ref	PRIMARY	PRIMARY	4	db_454rqctf2.E.DeptID	1	100	null

11. EXPLAIN -Three-Way JOIN

EXPLAIN

```

SELECT E.EmpName,
       D.DeptName,
       P.ProjectName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID
INNER JOIN Project P
ON E.ProjectID = P.ProjectID;

```

MySQL - db_454rqctf2											
EXPLORER Search... TABLES department departments employee employees leaves project salaries FUNCTIONS GetAverageSalaryOfDepartment PROCEDURES											
<pre> 197 EXPLAIN 198 SELECT E.EmployeeName, 199 D.DepartmentName, 200 P.ProjectName 201 FROM Employee E 202 INNER JOIN Department D 203 ON E.DeptID = D.DeptID 204 INNER JOIN Project P 205 ON E.ProjectID = P.ProjectID; </pre>											
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	D	null	index	PRIMARY	DeptName	202	null	5	100	Using index
1	SIMPLE	E	null	ref	DeptID,ProjectID	DeptID	5	db_454rqctf2.D.DeptID	6	100	Using where
1	SIMPLE	P	null	eq_ref	PRIMARY	PRIMARY	4	db_454rqctf2.E.ProjectID	1	100	null

12. Comparison of Execution Plans

The **EXPLAIN** command can be used to compare the execution plans of different SQL queries.

The execution plan helps in understanding how the database processes a query. It can also help identify whether indexes are being used and how many rows are estimated to be examined.

Query	Purpose	Execution Plan Information
INNER JOIN	Combines matching employee and department records	Shows tables accessed and join method
LEFT JOIN	Returns all employees with matching departments	Shows left table and matching access
Three-Way JOIN	Combines three related tables	Shows access and join order for three tables

14. Result

The Employee-Department-Project schema was successfully used to implement INNER JOIN, LEFT JOIN, self-join, three-way join, correlated subqueries, EXISTS, simulated

INTERSECT, and simulated EXCEPT. The EXPLAIN command was also used to examine and compare the execution plans of SQL queries.

15. Learning Outcomes

After completing this experiment, we are able to:

1. Use different types of SQL joins.
2. Compare records within the same table using self-join.
3. Combine three related tables using a three-way join.
4. Write and understand correlated subqueries.
5. Use EXISTS and NOT EXISTS.
6. Simulate INTERSECT and EXCEPT operations.
7. Analyze SQL execution plans using EXPLAIN.

16. Conclusion

This experiment provided practical knowledge of advanced SQL queries. Different types of joins, subqueries, EXISTS, simulated set operations, and EXPLAIN were successfully implemented on the Employee-Department-Project database schema.